Очереди и пределы производительности

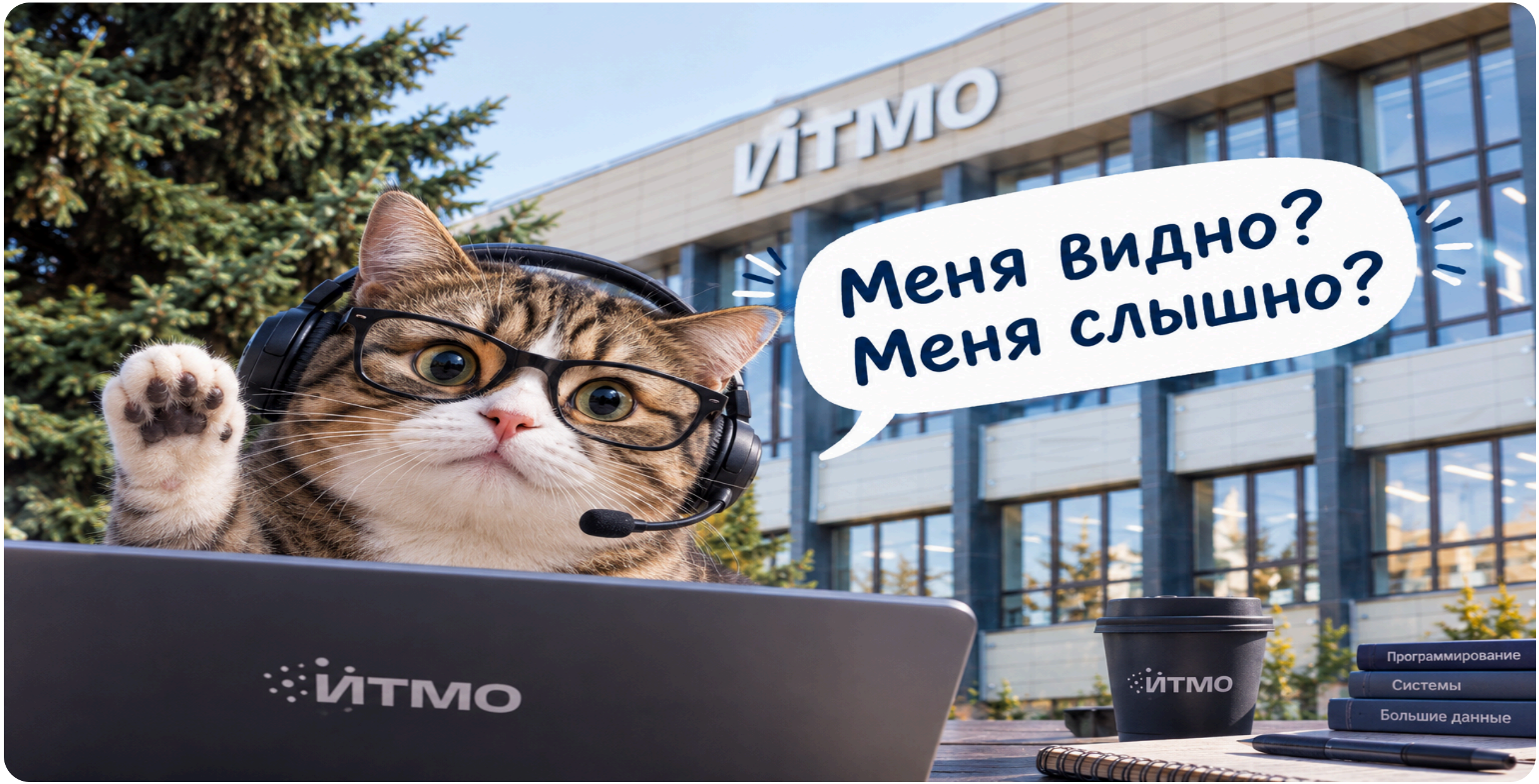
Семён Мартынов, Кирилл Дикалин, Георгий Семёнов

ИНСТИТУТ ПРИКЛАДНЫХ КОМПЬЮТЕРНЫХ НАУК

Национальный исследовательский университет ИТМО

8 октября 2026

Проверка связи



Цели лекции

1. **Объяснить**, почему при росте нагрузки в системе возникают очереди — даже когда ресурс загружен не на 100%.
2. **Связать** интенсивность потока λ, производительность ресурса μ и задержку W количественно.
3. **Оценивать** число одновременно обрабатываемых запросов и размеры пулов (закон Литтла).
4. **Предсказывать** поведение системы вблизи насыщения: на каких моделях ($M/M/1$, $M/M/c$, $M/M/c/K$) и какими формулами.
5. **Понимать**, где модели перестают работать: пачечный трафик, ретраи, тяжёлые хвосты.

Вопрос лекции: **какую загрузку мы можем себе позволить и чем за неё платим?**

План лекции

0. Уточнение требований курса
1. Поток работы и обслуживание: λ, μ, ρ, насыщение
2. Закон Литтла: $L = \lambda W$ и его практическое применение
3. Очереди: $M/M/1$, $M/M/c$, $M/M/c/K$, формулы Эрланга
4. Реальный трафик: всплески, ретраи, очередь как буфер

Уточнение требований курса

Критерии оценки:

- **10 квизов** -- 20 баллов // только на лекциях, но с возможностью досдать
- **Домашние/проектные задания** -- 40 баллов // по направлениям
- **Финальный экзамен** -- 40 баллов // по направлениям

Шкала перевода баллов в оценку:

Оценка	Баллы
5	≥ 90
4	≥ 75
3	≥ 60
2	< 60

Загадка

Сервер обрабатывает запрос в среднем за **10 мс** → его предел $\mu = 100$ RPS.
Средний входящий поток $\lambda = 90$ RPS → загрузка $\rho = 90\%$.

Сколько в среднем проведёт в системе запрос?

- Запросы приходят **строго раз в 11.1 мс**, обработка ровно 10 мс → очереди нет, $W = 10$ мс.
- Запросы приходят **случайно**, время обработки случайное (среднее 10 мс) → $W = 100$ мс, а $p99 \approx 460$ мс.

Средние те же. Загрузка та же. Откуда в 10 раз больше?

2.1 Откуда берётся очередь

Два принципиально разных источника очередей:

	Перегрузка	Изменчивость
Условие	$\lambda \geq \mu\,(\rho \geq 1)$	$\lambda < \mu\,(\rho < 1)$
Нужна ли случайность?	**Нет**, достаточно арифметики	**Да**: неравномерный приход или разное время обслуживания
Что происходит	Очередь растёт **без предела**	Очередь **конечна в среднем**, но случайно «надувается»
Лечится	Ёмкостью, отбрасыванием нагрузки	Запасом, снижением изменчивости

Сначала разберём первое (детерминированно), потом второе (теория очередей).

2.1 Система обслуживания



- λ — **интенсивность прихода** (arrival rate): сколько запросов/с поступает
- μ — **скорость обслуживания** (service rate): сколько запросов/с обработает *один* занятый сервер
- Между ними — **очередь**: место, где работа ждёт, пока ресурс занят

2.1 Обозначения лекции

Символ	Смысл	Связи
λ	интенсивность прихода, запр/с	
S, μ	время обслуживания; скорость $\mu = 1/\mathbb{E}[S]$	
c	число одинаковых серверов	
$a = \lambda/\mu$	предложенная нагрузка (offered load), **эрланги**	среднее число занятых серверов
$\rho = \lambda/(c\mu)$	загрузка (utilization) одного сервера	устойчивость: $\rho < 1$
L, L_q	среднее число запросов в системе / в очереди	
W, W_q	среднее время в системе / в очереди	$W = W_q + \mathbb{E}[S]$
X	пропускная способность (throughput)	при $\rho < 1$: $X = \lambda$

2.1 Загрузка и насыщение

Закон загрузки (utilization law): $U = X \cdot S$ *

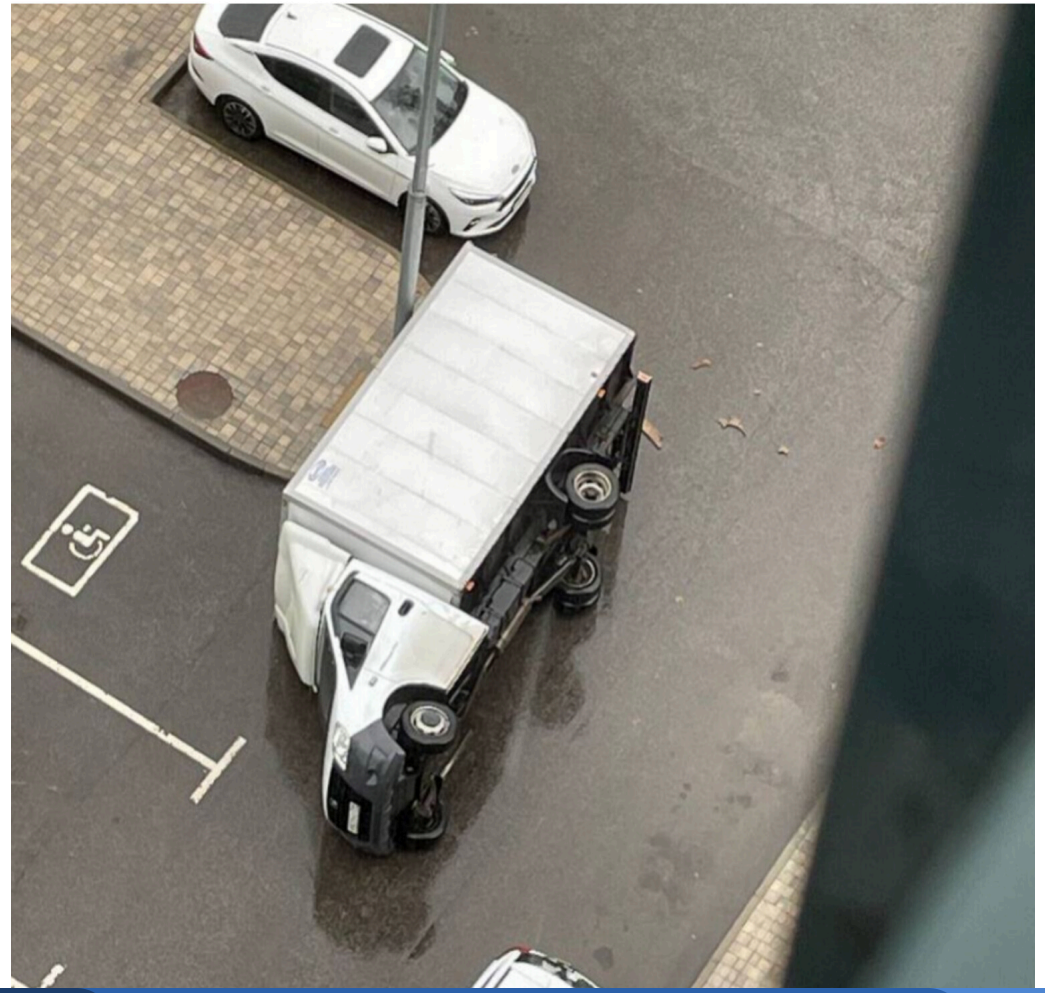
Пример: 1 ядро CPU обрабатывает $X = 300$ запр/с по $S = 2$ мс → $U = 300 \cdot 0.002 = 60\%$.

	Что измеряет	Типичная метрика
Utilization	доля времени, когда ресурс **занят**	busy time / окно
Saturation	есть ли работа, которая **ждёт** ресурс	run queue, queue depth, pending

- Это буквы **U** и **S** из метода USE (лекция 1)
- Загрузка 60% **за минуту** не исключает насыщения **в течение 100 мс** (ловушка окна усреднения)
- Насыщение — это уже не про «занят», а про «не успевает»

* Формула для одного сервера (одного ядра).
Для c одинаковых серверов: $U = X \cdot S/c = \rho$, а $X \cdot S = a$ — среднее число занятых серверов (эрланги).

всё, я больше не вывожу



2.1 Перегрузка: арифметика очереди

Пусть $\lambda = 1100$ запр/с, $\mu = 1000$ запр/с. Никакой случайности, чистая арифметика:

$$Q(t) \approx (\lambda - \mu)\, t, \qquad W(t) \approx \frac{Q(t)}{\mu} = \frac{\lambda - \mu}{\mu}\, t$$

Время t	Очередь Q	Ожидание вновь пришедшего
10 с	1 000	1 с
60 с	6 000	6 с
10 мин	60 000	1 минута

Превышение всего на **10%** → задержка растёт **линейно и бесконечно**.

Стационарного режима нет. Если очередь неограничена: память кончится раньше, чем терпение клиентов.

2.2 Закон Литтла

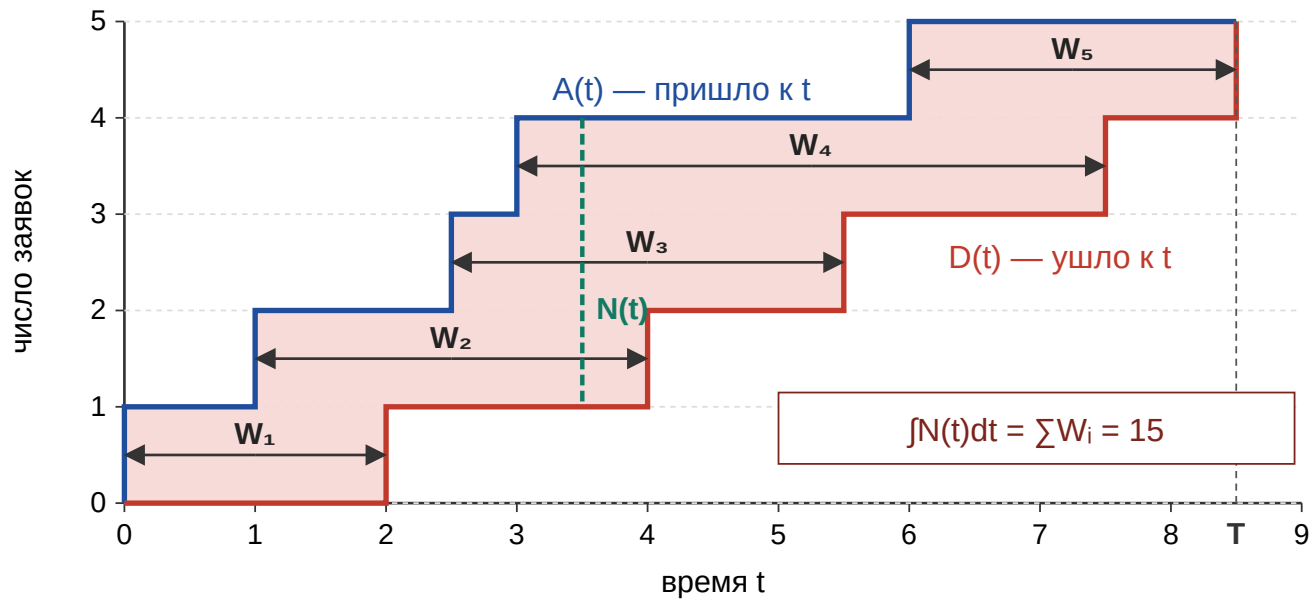
$$L = \lambda W$$

L — среднее число запросов **в системе**, λ — средняя интенсивность потока, W — среднее время **в системе**.

- Справедлив для **любого** процесса прихода, **любого** распределения времени обслуживания, **любой** дисциплины (FIFO, LIFO, приоритеты)
- Нужно лишь: система **устойчива**, и средние существуют
- Не требует пуассоновости, экспоненциальности, независимости: поэтому он — главный рабочий инструмент

J. D. C. Little, «A Proof for the Queuing Formula: L = λW», Operations Research 9(3), 1961.

2.2 Почему это верно



Система (FIFO) пуста при $t = 0^-$ и $t = T = 8.5$.

Площадь между $A(t)$ и $D(t)$ считаем двумя способами:

- **по вертикали**: срезы $N(t)$, площадь $\int_0^T N(t)\,dt$
- **по горизонтали**: полосы W_i, площадь $\sum_i W_i$

$$\bar{L} = \frac{1}{T}\int_0^T N(t)\,dt = \frac{1}{T}\sum_i W_i = \underbrace{\frac{n}{T}}_{\lambda} \cdot \underbrace{\frac{\sum_i W_i}{n}}_{\bar{W}} = \lambda\,\bar{W}$$

Здесь $n = 5, \sum_i W_i = 15$: $\bar{L} = \dfrac{15}{8.5} \approx 1.76 \approx 0.588 \cdot 3.0$ ✓.

Если в T система не пуста, то $\int_0^T N\,dt = \sum_i W_i - \Delta(T)$, где $\Delta(T)$ – остаток времени заявок после T.
$L = \lambda W$ при $T \to \infty$, если $\lambda = \lim n/T$ и $W = \lim \bar{W}$ конечны и $\Delta(T) = o(T)$.

2.2 Закон Литтла: границы системы

Граница выбирается **произвольно**, главное — применять её **одинаково** ко всем трём величинам:

Что считаем	L	W
вся система	$L = \lambda W$	время в системе
только очередь	$L_q = \lambda W_q$	время ожидания
только серверы	$L_s = \lambda \mathbb{E}[S] = a$	время обслуживания
один сервис в цепочке	in-flight запросов сервиса	его собственная задержка

Последняя строка: **среднее число занятых серверов равно** $a = \lambda / \mu$ (потому a измеряют в эрлангах).

Если часть запросов отбрасывается, в λ идёт **интенсивность принятых** λ_{eff}.

2.2 Чего Литтл не говорит

- Он про **средние**. Для перцентилей *простого* аналога нет: хвост W не получить из хвоста L умножением на λ.

  Распределённая версия закона существует, но только для FIFO без обгонов
- Он требует **устойчивости**: пока очередь растёт, $\lambda \neq$ интенсивность ухода, и равенство нарушено
- Граница должна быть **одной и той же**: «RPS на входе» и «время в БД» нельзя смешивать
- Окно измерения должно быть **длиннее** типичного W и не пересекать смену режима нагрузки
- Он ничего не говорит о **причинах**: $L = \lambda W$ — тождество, а не модель поведения

Литтл не предсказывает W. Он **связывает** три величины: две измерили — третью получили.

2.2 Практика: сколько запросов «в полёте»

Сценарий	λ	W	$L = \lambda W$
API-шлюз	1000 RPS	50 мс	**50** запросов в обработке
Пул соединений к БД	2000 RPS	5 мс	**10** соединений в среднем
То же, БД «просела»	2000 RPS	500 мс	**1000** → пул исчерпан
WebSocket / сессии	50 новых/с	240 с	**12 000** соединений

Память: 500 запросов в полёте × 2 МБ на запрос = **1 ГБ** только на состояние запросов.

Размер пула «по среднему» — **нижняя** граница. Запас под разброс — тема §2.3 и §2.4.

2.2 Литтл как инструмент

1. Ёмкость от ограничения. Лимит пула L_{max} → максимум потока $\lambda_{max} = L_{max}/W$.
200 потоков, $W = 100$ мс → **2000 RPS**. Если смежный сервис замедлился, и $W = 1$ с → **200 RPS**: ёмкость упала в 10 раз при нулевой загрузке CPU.

2. Диагностика. Метрика in-flight (L) и RPS (λ) дают $W = L/\lambda$. Не сходится со средним из гистограммы → разные границы измерения или нестационарность.

3. Каскад. При фиксированном λ рост W **поднимает** L: потоки, соединения, память кончаются → медленный сервис «заражает» вызывающих.

Лекция 1: «рост latency при том же RPS поднимает число одновременных запросов». Теперь это формула.

2.2 Литтл повсюду: мостики

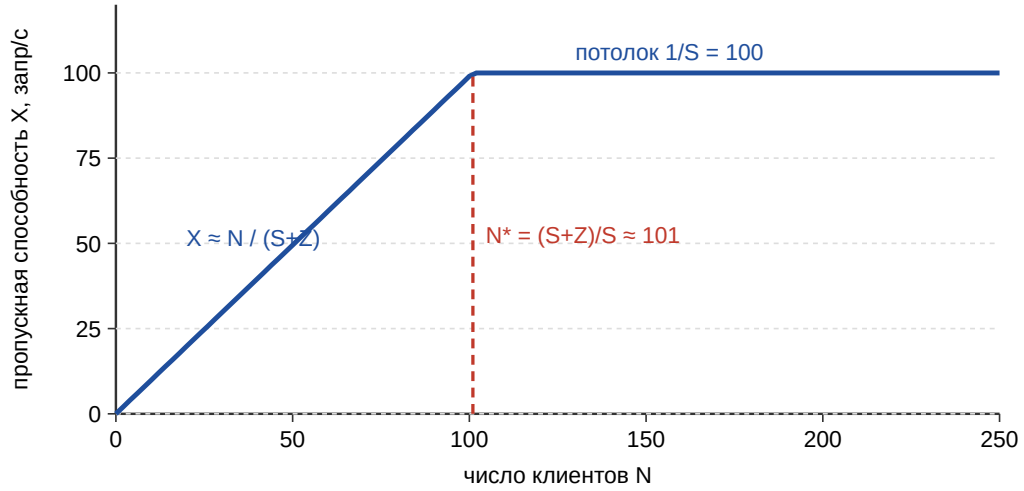
Область	L	λ	W
Сети	байт «в трубе»	пропускная способность канала	RTT
Архитектура ЭВМ	непрерывные обращения к памяти (outstanding misses)	пропускная способность памяти	латентность памяти
Разработка (Kanban)	незавершённая работа (WIP)	производительность команды	lead time
Базы данных	активные соединения	TPS	время транзакции

- **Сети:** bandwidth-delay product. 1 Гбит/с × 20 мс = 20 Мбит ≈ **2.5 МБ** — столько нужно окно TCP, чтобы загрузить канал
- **ЭВМ:** ~25 ГБ/с × ~80 нс ≈ 2 КБ ≈ 32 строки кэша одновременно (порядок величин). Поэтому нужна memory-level parallelism
- **Kanban:** уменьшаем WIP → при той же производительности сокращается lead time

2.2 Литтл и замкнутая модель

В замкнутой модели (лекция 1) N клиентов, каждый «думает» время Z, потом ждёт ответ R. Литтл для всего цикла: $N = X\,(R + Z)$, откуда

$$R = \frac{N}{X} - Z, \qquad X(N) \le \min\left\{\frac{N}{S+Z},\ \frac{1}{S}\right\}$$



$S = 10$ мс, $Z = 1$ с: колено при $N^* \approx 101$. При $N = 200$: $R \ge NS - Z = 1$ с, то есть **100×** от S.

Операционный анализ: Denning, Buzen (1978).

2.3 Где живут очереди

Ресурс	Очередь	Метрика
CPU	run queue планировщика	load average, run queue length
Сеть (вход)	accept queue, `listen backlog`	`Recv-Q`, переполнения backlog
Приложение	очередь задач пула потоков, event loop	queue size, in-flight
Пул соединений	ожидание свободного соединения	pool wait time
Диск	очередь запросов к устройству	queue depth, `await`
Сетевой стек	буферы NIC, qdisc	drops, `tx queue`
БД	ожидание блокировок	lock wait
Брокер	непрочитанные сообщения	consumer lag

Очереди бывают **явные** (вы их создали) и **неявные** (ОС, драйвер, библиотека). Неявная очередь не исчезает оттого, что её не видно.

Неограниченная ($K = \infty$)

- никто не потерян
- задержка растёт **без предела**
- память растёт до **OOM**
- перегрузка видна поздно: когда клиенты уже отвалились по таймауту



Ограниченная (ёмкость K)

- задержка $\lesssim K \cdot S$
- память предсказуема
- нужна **политика переполнения**: отказать (fail fast, 429/503), блокировать производителя (backpressure), вытеснить старое/новое
- платим **потерями**



2.3 Дисциплины обслуживания

- **FIFO** — «по справедливости»; при перегрузке выдаёт ответы **уже ненужные**: клиент ушёл по таймауту
- **LIFO** — свежие запросы первыми: при перегрузке большая часть успевает уложиться в таймаут, старые — приносятся в жертву
- **Adaptive LIFO**: FIFO в норме, переключение на LIFO, когда очередь растёт (Facebook, *Fail at Scale*, Maurer, 2015)
- **Приоритеты**: сначала важное. Цена: низкий приоритет может **голодать**
- **Active queue management** (CoDel): управляем не длиной очереди, а **временем ожидания в ней**

Для дисциплин, не знающих размера запроса (FIFO, LIFO, случайный выбор), средние L и W **одинаковы**, а вот **хвост** различается, и сильно.

2.3 Нотация Кендалла

A / S / c / K — процесс прихода / распределение времени обслуживания / число серверов / ёмкость системы

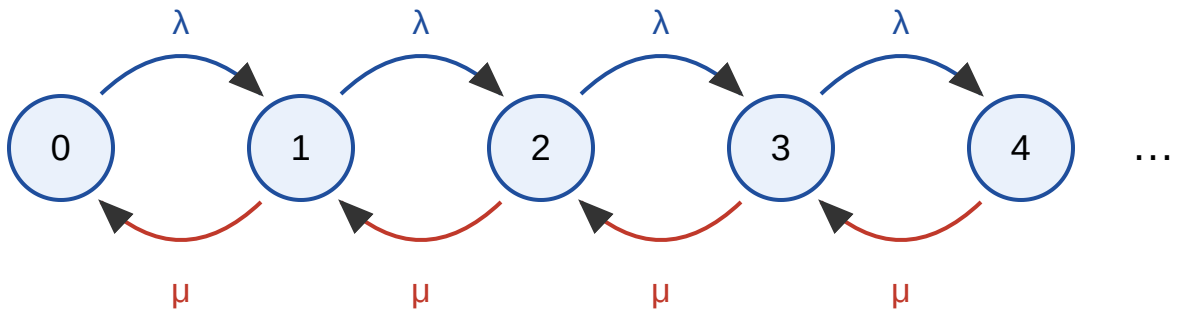
Символ	Значение
M (?!!)	Markovian: пуассоновский приход, ИЛИ (!) экспоненциальное обслуживание
D	детерминированное
G	произвольное (general)
c	число серверов
K	**полная** ёмкость (в очереди и на серверах); по умолчанию ∞

Примеры: $M/M/1$, $M/D/1$, $M/G/1$, $M/M/c$, $M/M/c/K$, $G/G/c$.

Модель — не истина, а **инженерный инструмент**: порядок величин и качественные зависимости. Если модель говорит «в 10 раз хуже», этому стоит верить, а если «на 3% хуже» — нет.

2.3 M/M/1: модель

Допущения: пуассоновский поток λ; экспоненциальное обслуживание со средним $1/\mu$; один сервер; бесконечная очередь FIFO.



Состояние — число запросов n в системе. В равновесии поток вверх = потоку вниз (*детальный баланс*):

$$\lambda\,\pi_n = \mu\,\pi_{n+1} \;\Rightarrow\; \pi_n = \rho^{\,n}\,\pi_0, \qquad \rho = \frac{\lambda}{\mu}$$

Нормировка $\sum_{n\ge 0}\pi_n = 1$: $\pi_0 \sum_{n\ge 0}\rho^{\,n} = \frac{\pi_0}{1-\rho} = 1 \;\Rightarrow\; \pi_0 = 1-\rho,\ \pi_n = (1-\rho)\,\rho^{\,n}$

Ряд сходится **только** при $\rho < 1$.

Иначе нормировать нечего: стационарного распределения нет.

2.3 M/M/1: метрики

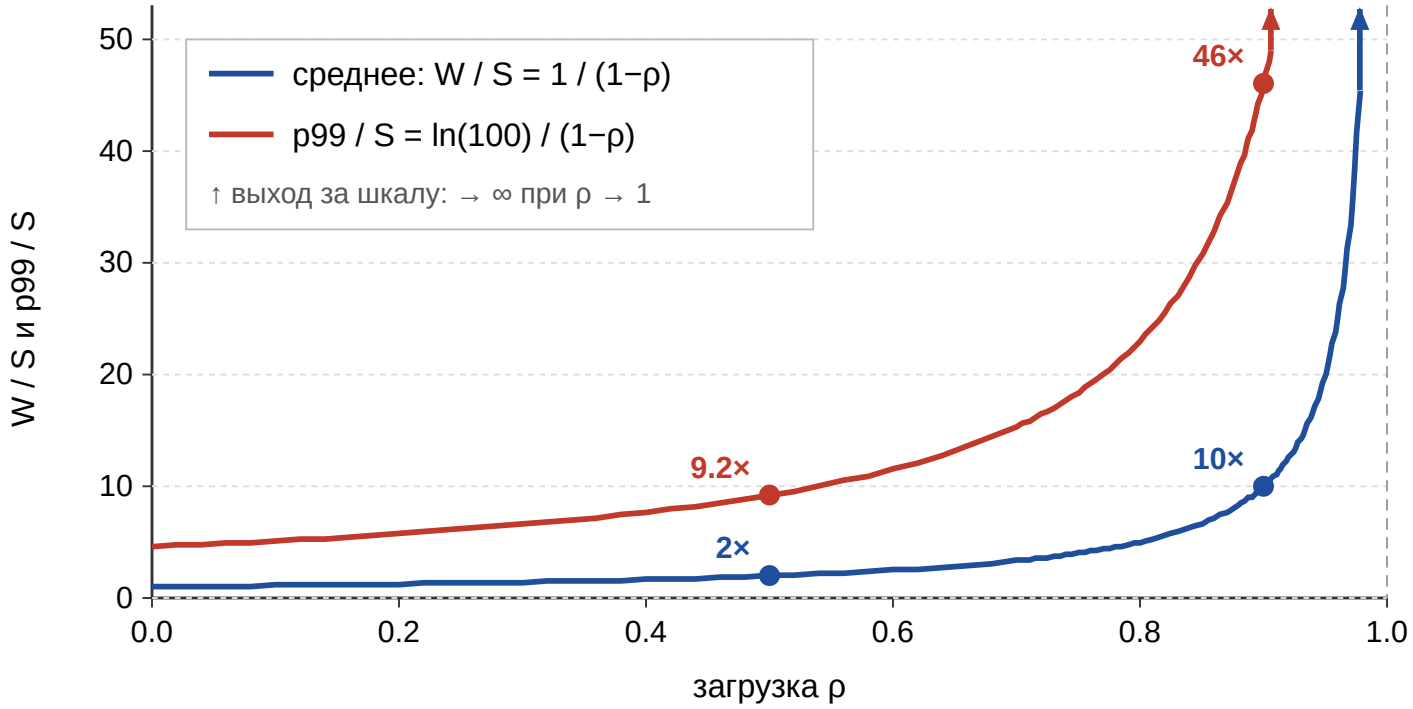
Величина	Формула
Число в системе	$L = \sum n\pi_n = \dfrac{\rho}{1-\rho}$
Число в очереди	$L_q = \dfrac{\rho^2}{1-\rho}$
Время в системе	$W = \dfrac{1}{\mu-\lambda} = \dfrac{S}{1-\rho}$
Ожидание	$W_q = \dfrac{\rho}{1-\rho}\,S$

Проверка Литтла: $\lambda W = \dfrac{\lambda}{\mu-\lambda} = \dfrac{\rho}{1-\rho} = L$ ✓

Распределение (FIFO): время в системе $T \sim \mathrm{Exp}(\mu-\lambda)$, откуда

$$P(T>t) = e^{-(\mu-\lambda)t}, \qquad T_p = -\ln(1-p)\cdot W \;\Rightarrow\; p99 \approx 4.6\,W,\; p99.9 \approx 6.9\,W$$

2.3 M/M/1: «хоккейная клюшка»



Лекция 1: «задержка растёт нелинейно». Теперь строго: при $S = 10$ мс и $\rho = 0.9$ среднее $W = 100$ мс, а **p99 ≈ 460 мс**. Это ответ на загадку.

2.3 M/M/1: цифры

Время выражено в единицах S (время обслуживания):

ρ	L	W/S	$p99/S$	при S = 10 мс: W → p99
0.5	1	2	9.2	20 → 92 мс
0.7	2.3	3.3	15.4	33 → 154 мс
0.8	4	5	23	50 → 230 мс
0.9	9	10	46	100 → 461 мс
0.95	19	20	92	200 → 921 мс
0.99	99	100	461	1 с → 4.6 с

Перейти с 90% на 95% загрузки (+5 п.п.) — **удвоить** среднюю задержку.

2.3 Почему рост нелинейный

1. Производная. $\dfrac{dW}{d\rho} = \dfrac{S}{(1-\rho)^2}$: каждый следующий процент загрузки стоит дороже.

При $\rho = 0.5$: $4S$ на единицу ρ; при 0.9: $100S$; при 0.99: $10\,000S$.

2. Запас рассасывает очередь. Очередь убывает со скоростью $\mu - \lambda = \mu(1-\rho)$. Всплеск из B запросов рассасывается за $B/(\mu-\lambda)$. Мало запаса $\to$ «память» о всплеске живёт долго, а следующий всплеск накладывается на хвост предыдущего.

3. Случайное блуждание. Длина очереди — блуждание со **сносом** $-(\mu-\lambda)$, отражённое от нуля. Чем меньше снос, тем **дальше** от нуля блуждание заходит. При $\rho \to 1$ снос $\to 0$, средняя длина $\to \infty$.

2.3 Три рычага задержки

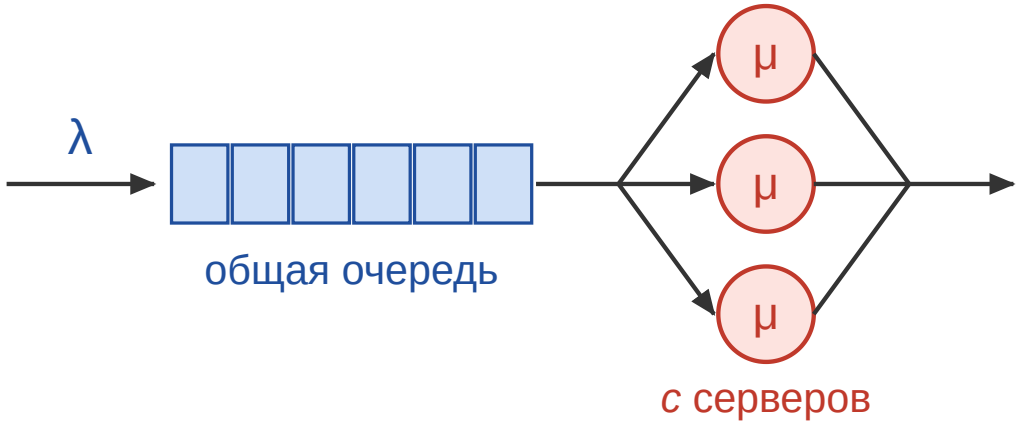
Формула Кингмана (приближение для $G/G/1$, асимптотически точное при $\rho \to 1$):

$$W_q \approx \underbrace{\frac{\rho}{1-\rho}}_{\text{загрузка}} \cdot \underbrace{\frac{c_a^2 + c_s^2}{2}}_{\text{изменчивость}} \cdot \underbrace{\mathbb{E}[S]}_{\text{масштаб}}, \qquad c^2 = \frac{\mathrm{Var}}{\mathrm{mean}^2}$$

Система	c_a^2	c_s^2	Множитель
$D/D/1$	0	0	**0**, очереди нет
$M/D/1$ (точно, Поллачек–Хинчин)	1	0	**1/2**
$M/M/1$ (точно)	1	1	**1**
$M/G/1$, тяжёлый хвост	1	10	**5.5**

Загрузку снижает **железо**, изменчивость — **инженерия**: изоляция тяжёлых запросов в отдельный пул, лимиты на размер, таймауты.

2.3 M/M/c: несколько серверов



- c серверов, **одна общая** очередь: освободившийся сервер берёт следующий запрос
- предложенная нагрузка $a = \lambda/\mu$ (эрланги), загрузка $\rho = a/c$; устойчивость: $\rho < 1$

Формула Эрланга C — вероятность того, что запрос будет **ждать**:

$$\Theta = \frac{a^c}{c!\,(1-\rho)} \qquad C(c,a) = \frac{\Theta}{\sum_{k=0}^{c-1} \frac{a^k}{k!} + \Theta}$$

Для $c = 1$: $a = \rho$, $\Theta = \frac{\rho}{1-\rho}$ и $C = \rho$ — как в $M/M/1$.

2.3 M/M/c: метрики

$$W_q = \frac{C(c,a)}{c\mu-\lambda}, \qquad L_q = C(c,a)\,\frac{\rho}{1-\rho}, \qquad P(W_q > t) = C(c,a)\,e^{-(c\mu-\lambda)\,t}$$

Проверка: $c = 1 \Rightarrow C = \rho$ и получаем формулы $M/M/1$ ✓.

Факториалы переполняются. Считаем рекуррентно (формула Эрланга B, далее перейдём к C):

```
def erlang_b(c, a):           # a = λ/μ, в эрлангах
    b = 1.0
    for k in range(1, c + 1):
        b = a * b / (k + a * b)
    return b

def erlang_c(c, a):           # требуется a < c
    rho = a / c
    b = erlang_b(c, a)
    return b / (1 - rho * (1 - b))
```

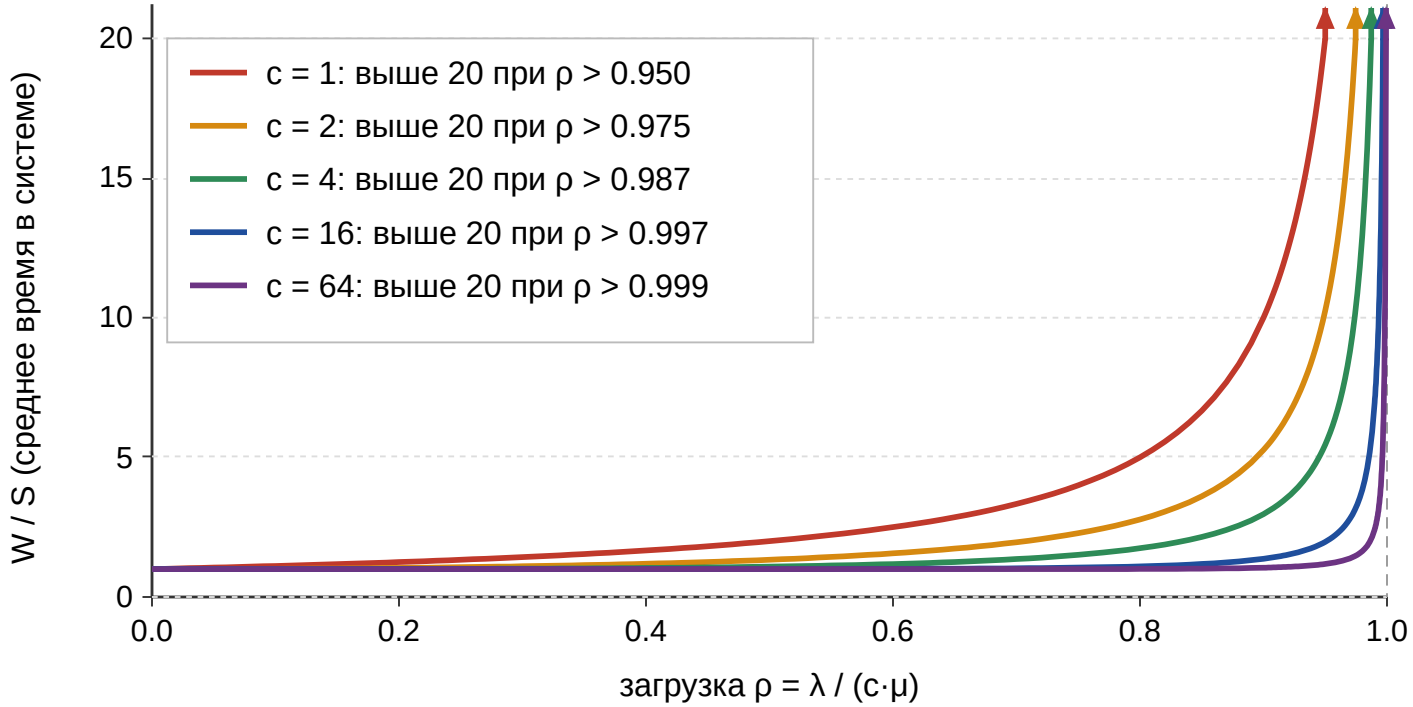
2.3 Эффект масштаба: одна очередь на всех

Загрузка **одна и та же**, $\rho = 0.9$, время обслуживания S. Растёт только число серверов c:

c	a	$P(\text{ждать}) = C$	W_q / S
1	0.9	0.90	9.0
4	3.6	0.79	2.0
8	7.2	0.70	0.88
16	14.4	0.59	0.37
64	57.6	0.31	0.049

Разброс: 9.0 против 0.049 — в **180 раз** при той же ρ. Крупные общие пулы переносят высокую загрузку гораздо легче мелких.

2.3 Эффект масштаба: график



«Колено» сдвигается вправо с ростом c.

При $\rho = 0.9$: $c = 1$ даёт $W = 10\,S$, а $c = 16$ — всего $\approx 1.4\,S$.

2.3 Одна общая очередь или по очереди на сервер

8 серверов, $\rho = 0.9$. Два способа раздать работу:

Схема	Модель	W_q / S
Общая очередь, свободный сервер берёт следующий	$M/M/8$	**0.88**
Случайная раздача по серверам, у каждого своя очередь	$8 \times M/M/1$	**9.0**

Случайное разбиение пуассоновского потока снова пуассоновское, поэтому каждая очередь — это $M/M/1$ с $\rho = 0.9$. **Разница в 10 раз** только от организации очереди.

Часть запросов ждёт у занятого сервера, пока соседний простаивает. Мостик к балансировке (лекция 5): *power of two choices* (Mitzenmacher, 2001) возвращает почти весь выигрыш, спрашивая у **двух** случайных серверов, кто короче.

2.3 Какую загрузку можно себе позволить

Требование к хвосту: $p99 \le k \cdot S$. Для $M/M/1$: $\dfrac{\ln 100}{1-\rho} \le k \;\Rightarrow\; \rho \le 1 - \dfrac{4.6}{k}$

Макс. загрузка ($M/M/c$, расчёт по распределению времени в системе):

c	1	2	4	8	16	64
$p99 \le 10\,S$	54%	76%	88%	94%	97%	99%
$p99 \le 20\,S$	77%	88%	94%	97%	98.5%	99.6%

- Нет универсального «не больше 70%». Целевая загрузка **выводится из SLO** и зависит от c
- Один сервер с требованием $p99 \le 10S$ нельзя держать выше **54%**
- Это **оценки сверху**: модель идеальна (пуассон, FIFO). Реальный трафик — §2.4

2.3 Квадратный корень

Сколько серверов нужно, чтобы вероятность ожидания $C(c, a) \leq 10\%$?

Нагрузка a	1	10	100	1000	10 000
Серверов c	3	16	115	1046	10 143
Загрузка $\rho = a/c$	33%	63%	87%	96%	98.6%
$(c - a)/\sqrt{a}$	2.0	1.9	1.5	1.45	1.43

Правило квадратного корня (Halfin, Whitt, 1981): $c \approx a + \beta\sqrt{a}$.

Запас растёт как $\sqrt{a}$, а не линейно, поэтому чем больше система, тем выше допустимая загрузка. Этот эффект — аргумент в пользу **общих** пулов.

2.3 Формула Эрланга B: системы без очереди

$M/M/c/c$: c каналов, **нет места** для ожидания. Занято всё → запрос **отбрасывается** (телефонная сеть, пул без ожидания, лимит соединений).

$$B(c, a) = \frac{a^c/c!}{\sum_{k=0}^{c} a^k/k!}$$

Формула верна и для **любого** распределения времени обслуживания ($M/G/c/c$). Эрланг, 1917.

Пул $c = 10$	$a = 5$	$a = 8$	$a = 10$
Доля отказов B	1.8%	12.2%	21.5%

При $a = c = 10$ отбрасывается более **каждого пятого** запроса.

2.3 M/M/c/K: конечный буфер

Полная ёмкость K: запрос, застающий K запросов в системе, **теряется**. Вероятность отказа P_K, **принятый** поток $\lambda_{\text{eff}} = \lambda(1 - P_K)$, и Литтл работает с ним: $W = L/\lambda_{\text{eff}}$.

Для $M/M/1/K$: $\pi_n = \dfrac{(1-\rho)\rho^n}{1-\rho^{K+1}}$, $P_K = \pi_K$. **Устойчива при любом** ρ, даже при $\rho > 1$.

$K = 10$, время в единицах S:

ρ	отказы P_K	λ_{eff}/μ	W/S принятых	W/S при $K = \infty$
0.5	0.05%	0.50	2.0	2.0
0.9	5.1%	0.85	4.6	10
1.0	9.1%	0.91	5.5	∞
1.2	19%	0.97	6.9	∞
2.0	50%	1.00	9.0	∞

2.3 Размер буфера: компромисс

Ограниченная очередь обменивает потери на предсказуемость.

- При $\rho = 0.9$ буфер $K = 10$ даёт задержку 4.6 S вместо 10 S, цена — **5% отказов**
- При $\rho = 2$ пропускная способность $\approx \mu$ при любом K. А **задержка** принятых $\approx K \cdot S$:

K	отказы	пропускная способность	W/S
10	50%	$\approx \mu$	9
50	50%	$\approx \mu$	49

Большой буфер **не добавил** обработанных запросов, но **в 5 раз** увеличил ожидание. Для сетей этот эффект называют *bufferbloat*.

Размер буфера выбирают по **допустимой задержке**: $K \approx W_{\max}/S$, а не «чтобы побольше влезло».

2.3 Перегрузка: полезная работа и таймауты

Неограниченная FIFO-очередь, $\lambda = 1100$, $\mu = 1000$, таймаут клиента **1 с**:

- ожидание нового запроса $\approx 0.1\,t$ секунд $\to$ при $t \geq 10$ с он **не дождётся обработки**
- сервер загружен на 100%, но обрабатывает запросы, **от которых клиент уже отказался**
- **Goodput** (полезная пропускная способность) $\to 0$ при полной загрузке CPU

Ещё хуже: клиенты **повторяют** запросы, нагрузка растёт, система не восстанавливается даже после спада исходной нагрузки: **метастабильный сбой**.

Лекарства (лекция 7): ограниченная очередь, load shedding, LIFO, отсечение просроченных запросов (deadline propagation), backpressure.

Bronson et al., «Metastable Failures in Distributed Systems», HotOS 2021; Huang et al., «Metastable Failures in the Wild», OSDI 2022.

2.3 Карта моделей

Модель	Допущения	Даёт	Применение
$M/M/1$	пуассон, экспонента, 1 сервер	$W = S/(1-\rho)$, хвост	оценки «на салфетке»
$M/G/1$	пуассон, любое S	формула Поллачека–Хинчина	влияние c_s^2
$G/G/1$	любые	Кингман (приближение)	влияние c_a^2 и c_s^2
$M/M/c$	общая очередь, c серверов	Эрланг C, $P(W_q > t)$	пулы потоков, воркеры
$M/M/c/c$	нет очереди	Эрланг B, доля отказов	лимиты соединений
$M/M/c/K$	конечный буфер	P_K, λ_{eff}	ограниченные очереди

Все они — **стационарные** модели: предполагают, что поток не меняется во времени. Реальный трафик устроен иначе.

2.4 Что такое пуассоновский поток

Процесс Пуассона с интенсивностью λ:

1. **Независимые приращения**: число запросов в непересекающихся интервалах независимо
2. **Стационарность**: λ не зависит от времени
3. **Ординарность**: два запроса в один момент не приходят

Следствия: интервалы между запросами $\sim \mathrm{Exp}(\lambda)$, $c_a^2 = 1$, а число запросов за окно t имеет $\mathrm{Var}\, N(t) = \mathbb{E}\, N(t)$.

Почему модель вообще разумна. Суперпозиция **многих независимых** разреженных потоков стремится к пуассоновскому (теорема Пальма–Хинчина): много независимых пользователей на коротких масштабах.

Все три свойства в реальном трафике **нарушаются**.

2.4 Почему трафик не пуассоновский

Механизм	Примеры	Что нарушено
Суточные и недельные циклы, события	распродажа, трансляция матча, утренний пик	стационарность
Синхронизация по часам	cron «каждую минуту в :00», истечение TTL кэша, рассылки	независимость
Каскады и fan-out	один запрос порождает десятки; страница → много API	ординарность
Обратная связь	ретраи, переподключения после сбоя	независимость
Тяжёлые хвосты размеров	размеры файлов, длина сессий, сложность запросов	$c_s^2 \gg 1$

Эмпирически: в сетевом трафике наблюдались **самоподобие** и корреляции на многих масштабах времени.

Leland, Taqqu, Willinger, Wilson (1994); Paxson, Floyd, «Wide-area traffic: the failure of Poisson modeling» (1995).

2.4 Как измерить «пачечность»

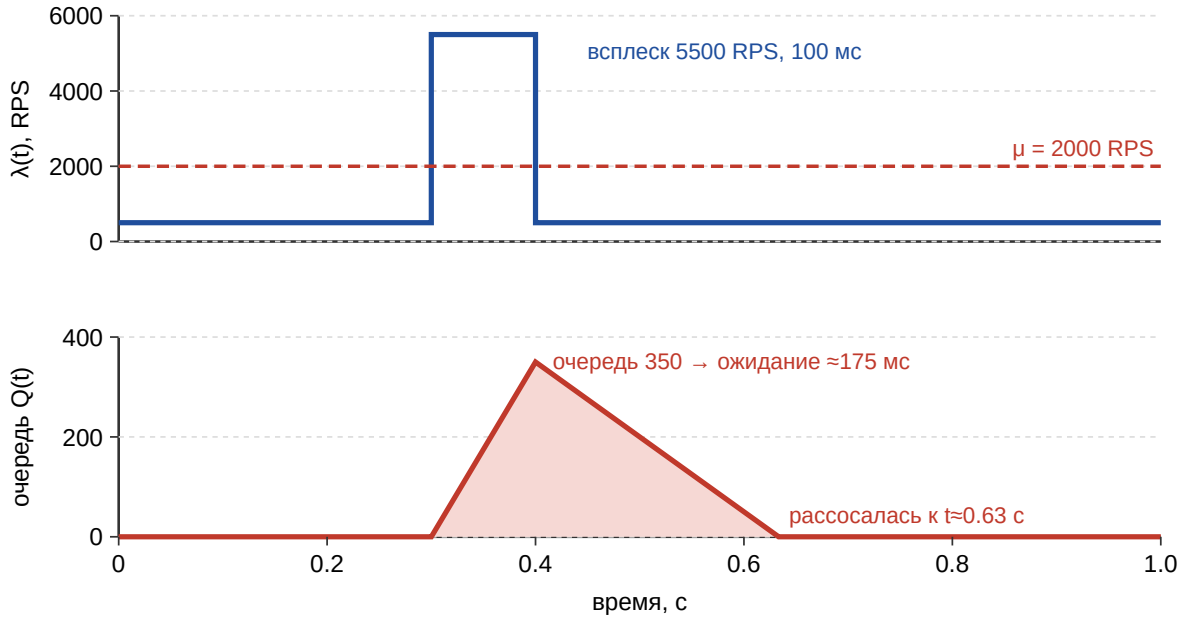
Index of dispersion for counts: $\mathrm{IDC}(t) = \dfrac{\operatorname{Var} N(t)}{\mathbb{E}\, N(t)}$, где $N(t)$ – число запросов в окне длины t.

- Пуассон: $\mathrm{IDC} = 1$ при **любом** t
- Пачки / всплески: $\mathrm{IDC} > 1$
- Самоподобный трафик: $\mathrm{IDC}(t)$ **растёт** с окном как t^{2H-1}, $H \in (0.5, 1)$ – параметр Хёрста

```
# ts — моменты прихода запросов, секунды
counts = np.bincount((ts - ts.min()).astype(int))   # запросов в секунду
print("IDC =", counts.var() / counts.mean())        # ≈ 1 для Пуассона
```

Осторожно: тренд (суточный цикл) тоже раздувает дисперсию. Считайте в **стационарных** окнах и для **нескольких** размеров t.

2.4 Пример: загрузка 50%, а очередь есть



За секунду: $450 + 550 = 1000$ запросов ($\rho = 50\%$ по минутной метрике). Но 100 мс интенсивность в 2.75 раза выше μ → очередь **350**, ожидание **≈175 мс** (ровно $350/\mu$).

2.4 Один ρ, разный трафик

$\rho = 0.7$, S — экспоненциальное, среднее 10 мс. Меняется **только** процесс прихода. Симуляция $2 \cdot 10^6$ запросов:

Процесс прихода	c_a^2	Ожидание, среднее	Время в системе, p99
Строго равномерный	0	8.7 мс	86 мс
Пуассон	1	23 мс	152 мс
Пачки по 10 одновременно	19	**173 мс**	**777 мс**

```
w = 0.0                                  # рекуррентность Линдли
for s, t in zip(service, interarrival[1:]):
    w = max(0.0, w + s - t)              # ожидание следующего запроса
```

Кингман для пачек: $\approx 23\,S$, симуляция: $17.3\,S$. Порядок верный, точность — не гарантируется.

2.4 Очередь как сглаживающий буфер

Сглаживание

- Буфер превращает **пик** в **среднее**: ёмкость можно ставить между λ_{avg} и λ_{peak}
- Для потока, ограниченного кривой $\sigma + \rho t$ (token bucket), сервер с $\mu \geq \rho$ даёт: очередь $\leq \sigma$, задержка $\leq \sigma / \mu$ (network calculus, Cruz, 1991)
- Буфер переживает всплеск, **не проваливая** запросы

Источник задержки

- Каждый «спасённый» запрос **платит ожиданием**: $L_q = \lambda W_q$
- Очередь **не добавляет ёмкости**: она переносит нагрузку во времени
- Буфер конечного размера заполняется за $K / (\lambda - \mu)$
- Умный буфер: ограничить и **время** ожидания, не только длину

Очередь сглаживает **дисперсию**, но не **дрейф**. Если средний $\lambda > \mu$, никакой буфер не поможет, он лишь отсрочит отказ.

2.4 Ретраи: обратная связь

Сбой → таймаут → **повтор**. Если каждая попытка проваливается с вероятностью p, итоговый поток:

$$\lambda_{\text{eff}} = \lambda\,(1 + p + p^2 + \dots) = \frac{\lambda}{1-p} \quad \text{(при неограниченных ретраях)}$$

Пример: $\mu = 1000$, исходный поток $\lambda = 700$ ($\rho = 0.7$), краткий сбой с $p = 0.5$:

Политика	Эффективный поток	ρ_{eff}
без ретраев	700	0.7
не более 3 повторов	$700 \cdot 1.875 = 1312$	**1.31**
неограниченные	$700/(1-0.5) = 1400$	**1.4**

Загрузка > 1 при исходных 70%. Хуже того, перегрузка **повышает** p → ещё больше ретраев. Положительная обратная связь.

Лечение (лекция 7): exponential backoff с jitter, лимит на ретраи (retry budget), circuit breaker.

2.4 Мостик: нагрузочные тесты

Всё сказанное меняет то, **как нужно тестировать** (лекция 9 и семинар 9):

- Модель **открытая** (лекция 1): клиенты не ждут ответа. Замкнутый генератор *сам себя тормозит* и скрывает очередь (coordinated omission)
- Приход — **не** равномерный и **не** чистый Пуассон: воспроизводите реальный профиль, пачки, пики, циклы
- Измеряйте **хвост** (p99, p99.9), а не среднее
- Меняйте нагрузку ступенями и ищите **колено**: где W начинает ускоряться
- Тест на $\rho = 0.7$ в «гладком» режиме **не доказывает**, что система выдержит $\rho = 0.5$ в пачечном

Чек-лист инженера

1. Считайте ρ по **самому узкому** ресурсу (USE) и в окнах, **соизмеримых** с длиной всплесков
2. Целевую загрузку **выводите из SLO по хвосту**, а не берите «70% по традиции»
3. По Литтлу: $\lambda_{max} = L_{max}/W$. Следите за **in-flight** и размерами пулов
4. Каждая очередь должна быть **ограничена**, а политика переполнения — **осознанна**
5. Снижайте **изменчивость**: изолируйте тяжёлые запросы, ставьте таймауты и лимиты
6. **Объединяйте** ресурсы в общие очереди, а не дробите на мелкие
7. Закладывайте в расчёт **пачки и ретраи**, а не только среднее

Итоги лекции

Идея	Формула
Закон Литтла (универсален)	$L = \lambda W$
Закон загрузки	$U = X \cdot S$
Задержка в $M/M/1$	$W = \dfrac{S}{1-\rho},\ p99 \approx 4.6\,W$
Три рычага	загрузка, изменчивость c^2, масштаб S
Общий пул лучше мелких	Эрланг C, правило $\sqrt{a}$
Ограниченная очередь	обменивает задержку на отказы: $W \lesssim KS$
Реальный трафик	пачки, циклы, ретраи → модель оптимистична

Следующая лекция — как жить при ρ, приближающемся к пределу: **масштабирование**.

Литература

- Harchol-Balter. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action* (Cambridge, 2013): основной учебник лекции
- Little. *A Proof for the Queuing Formula: $L = \lambda W$* (Operations Research, 1961)
- Lazowska, Zahorjan, Graham, Sevcik. *Quantitative System Performance* (1984): операционный анализ; Denning, Buzen (ACM Computing Surveys, 1978)
- Kleinrock. *Queueing Systems, Vol. 1: Theory* (Wiley, 1975)
- Kingman. *The single server queue in heavy traffic* (Proc. Cambridge Phil. Soc., 1961)
- Halfin, Whitt. *Heavy-traffic limits for queues with many exponential servers* (Operations Research, 1981)
- Paxson, Floyd. *Wide-area traffic: the failure of Poisson modeling* (IEEE/ACM ToN, 1995); Leland et al. *On the self-similar nature of Ethernet traffic* (1994)
- Mitzenmacher. *The Power of Two Choices in Randomized Load Balancing* (IEEE TPDS, 2001)
- Maurer. *Fail at Scale* (ACM Queue, 2015)
- Bronson et al. *Metastable Failures in Distributed Systems* (HotOS, 2021); Huang et al. *Metastable Failures in the Wild* (OSDI, 2022)

Обратная связь

Анонимная форма